

Reporte: Motor de Inferencia Lógica como Servicio

Implementación de Paradigmas de Programación mediante API REST

1. Introducción

El presente reporte detalla el diseño e implementación de un sistema de inferencia lógica basado en el paradigma declarativo, expuesto como un servicio web moderno. El proyecto integra la potencia de razonamiento de **Prolog** con la flexibilidad y escalabilidad de **Node.js**, permitiendo que aplicaciones externas realicen consultas lógicas complejas a través de protocolos estándar de internet (HTTP/JSON).

2. Explicación de Paradigmas de Programación Usados

El sistema se fundamenta en la convergencia de tres paradigmas esenciales:

Paradigma Lógico (Prolog): Se utiliza para definir la base de conocimiento. En lugar de programar una serie de pasos (algoritmos), se definen hechos y reglas. El motor de inferencia busca automáticamente las soluciones.

Paradigma Funcional (JavaScript): Empleado en el procesamiento de las peticiones. Se hace énfasis en el uso de funciones de primera clase y la transformación de datos (mapeo de respuestas de Prolog a JSON).

Paradigma Asíncrono: Node.js gestiona las consultas mediante un bucle de eventos, permitiendo que el servidor atienda múltiples peticiones simultáneamente mientras el motor lógico procesa las inferencias.

3. Arquitectura del Sistema

Componente	Tecnología	Función Principal
------------	------------	-------------------

Servidor Web	Express (Node.js)	Gestión de rutas y peticiones HTTP.
Motor Lógico	Tau Prolog	Ejecución de inferencias y evaluación de reglas.
Base de Conocimiento	Prolog (.pl)	Almacenamiento de hechos y reglas lógicas.
Cliente de Pruebas	Thunder Client	Validación de endpoints y formato JSON.

4. Ejemplo de Ejecución

Para validar el correcto funcionamiento, se configuró una regla de negocio donde se aplica una penalización si existe un contrato y un incumplimiento asociado. A continuación, se muestra la evidencia de una consulta exitosa para el sujeto "juan":

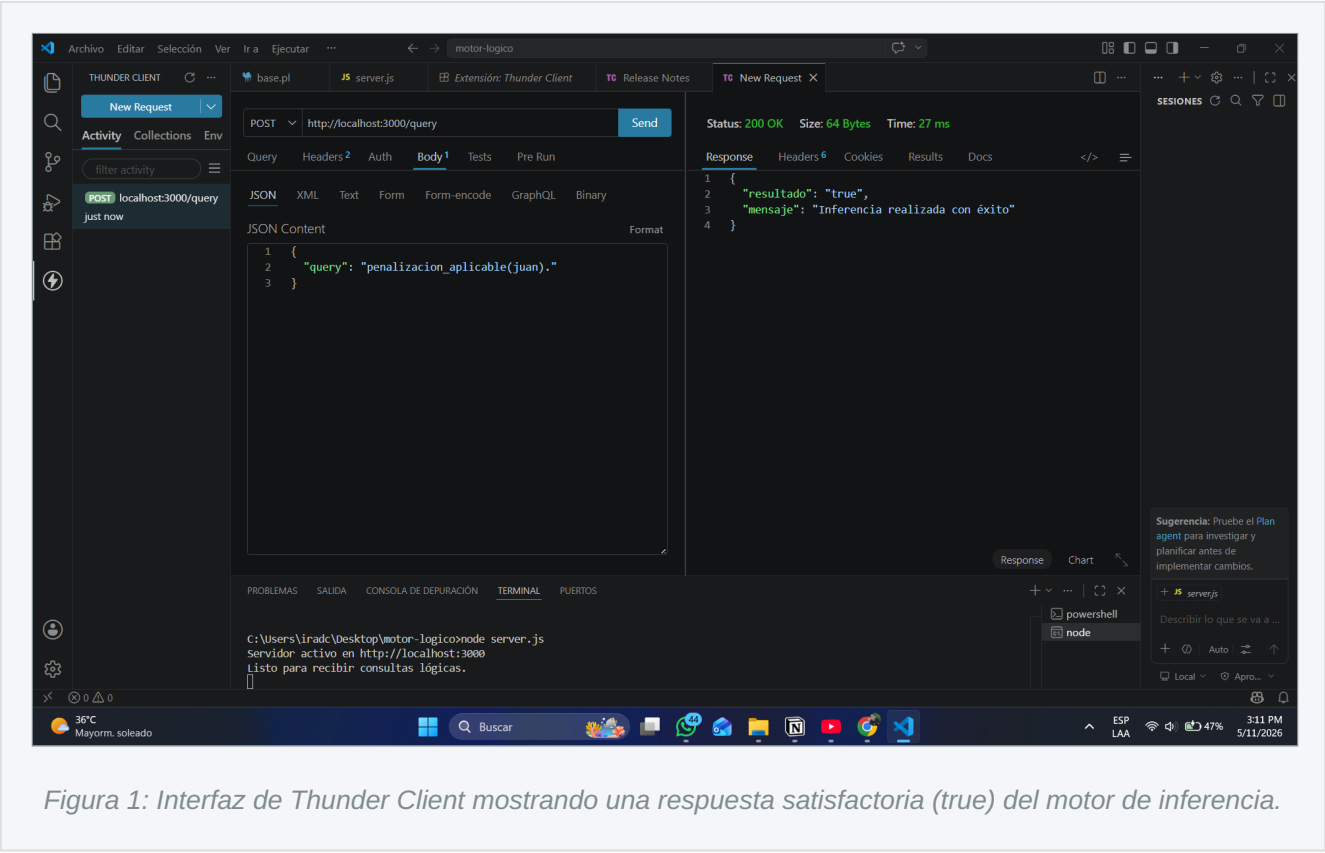


Figura 1: Interfaz de Thunder Client mostrando una respuesta satisfactoria (true) del motor de inferencia.

5. Conclusiones y Extensiones

La implementación demuestra que es posible desacoplar la lógica de negocio compleja (manejada en Prolog) de la capa de servicios (Node.js). Esto permite crear sistemas expertos que pueden ser consumidos por cualquier aplicación móvil o web.

Extensiones futuras:

- Carga dinámica de múltiples archivos de base de conocimiento.
- Implementación de seguridad JWT para restringir el acceso a la API.
- Uso de una base de datos relacional para persistir los hechos de Prolog.